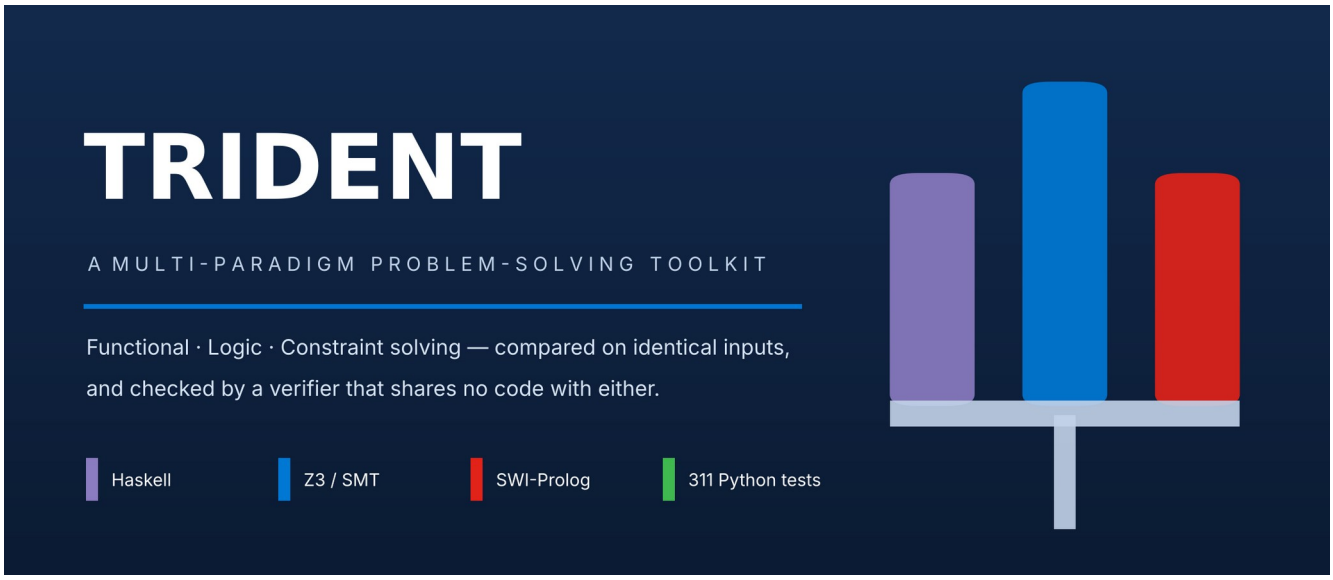


Technical & Engineering Report

A comparative study of functional, logic and constraint-based approaches to four classical computing problems

4	3	345	7
problem domains	programming paradigms	automated tests	green CI jobs

AUTHOR Elisha Benjamin Sreepalle

REPOSITORY github.com/ebenjamin12/trident

LICENCE MIT

ORIGIN MSc Advanced Programming (7017SCN), Coventry University — extended well beyond the assessed scope

WHAT'S INSIDE

01 **The question this project exists to answer** The research question, and the result that reverses

02 **Architecture** One pipeline, and why one function is written five times

03 **Independent verification** How a result earns the right to be called correct

04 **The four problem domains** Haskell ciphers · Z3 cryptarithms · TetraVex · Blocks World

05 **Results — and why the answer reverses** Measured comparisons, with the caveats kept in view

06 **Engineering: CI and two instructive defects** Two bugs a green test suite could not see

07 **Quality assurance** 345 tests, benchmark methodology

08 **Known limitations** Stated plainly

09 **Roadmap — the fifth domain** Cellular automata, and the prediction it would test

10 **Technology stack** Tools, versions and references

01 The question this project exists to answer

Trident began as coursework that asked for two problems implemented in Haskell and in Z3. It satisfied that brief, and then kept going, because the coursework could not answer the question the work actually raised:

> ◆ **The research question**
>
> When two very different solver paradigms can both attack the same problem, **which one actually wins — and is the answer a property of the tool, or of the problem?**

Answering that honestly takes more than two implementations. It needs identical inputs, an arbiter of correctness that cannot be fooled by a bug shared with the solver, and measurements that are taken rather than assumed. Trident supplies all three, and the headline finding is that the answer **reverses** between two problems attacked by the very same pair of engines.

Problem	Shape of the problem	Winner	Measured margin
TetraVex	One static constraint system	**Z3 (SMT)**	1.6× at 4×4, rising to 7.6× at 5×5
Blocks World	Search for a sequence of unknown length	**Prolog (IDA*)**	3.6×–15.8× on engine time

Same two solvers, opposite outcomes. The **shape** of the problem decides, not the quality of the tool — and demonstrating that with numbers rather than asserting it is the entire point of the repository.

Engineering objectives

- **Independent verification.** No result is trusted because the thing that produced it says so.
- **Reproducibility.** Every instance is seeded; every benchmark records its seeds, environment and raw measurements.
- **Honest measurement.** Startup cost separated from solve time; medians reported with observed ranges; no cross-engine ratios where the units differ.
- **Paradigm comparison as the deliverable**, not an incidental benchmark.

02 Architecture

The two newer domains share one layered structure. The arrows below are the *only* permitted import directions, and automated tests enforce the critical ones — this is a design rule with a failing test behind it, not a diagram-level aspiration.

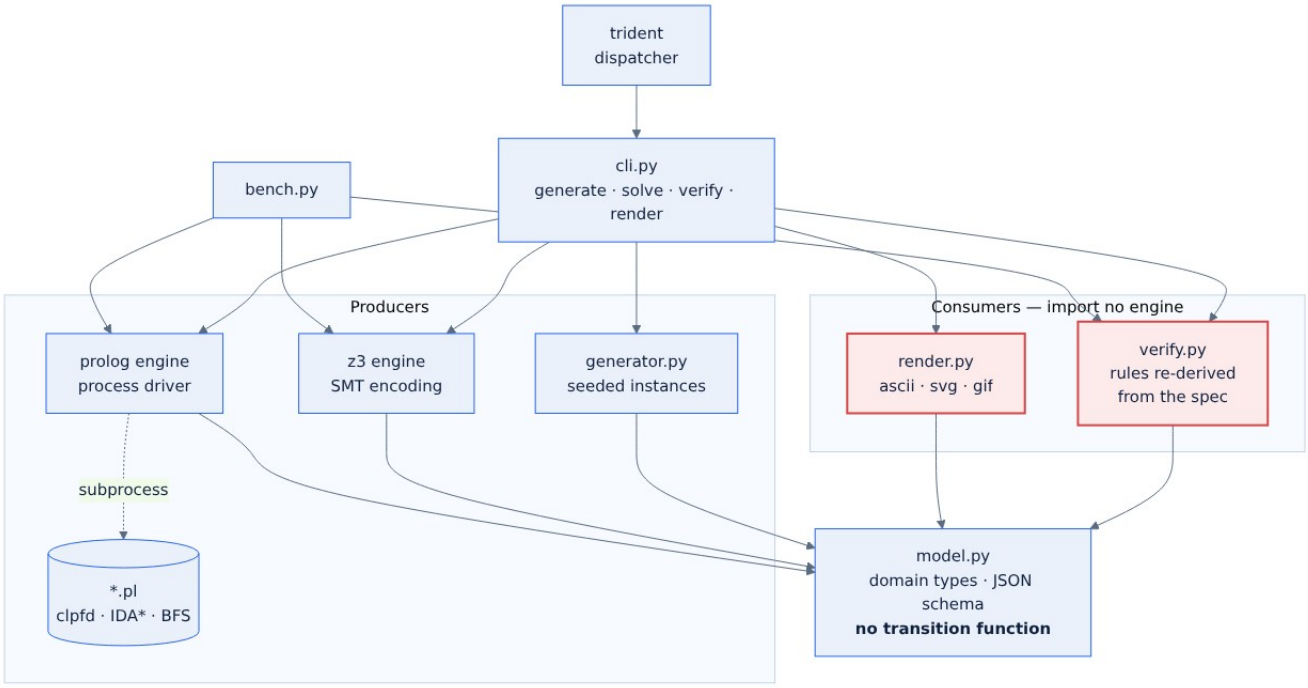


Figure 1. *The shared pipeline. Modules outlined in red are consumers: they import no engine. model.py deliberately contains no transition function.*

The transition function is implemented five times on purpose

"Apply an action to a state" exists five separate times in Blocks World: in the Prolog clauses, in the Z3 constraints, in the verifier, in a generator-local copy, and in the BFS test oracle. Consolidating them is the obvious refactor, and it would destroy the project's central correctness argument.

> ▲ **Why the duplication is load-bearing**
>
> If the verifier shared code with an engine, a bug in the shared part would be **invisible** — the engine would produce a wrong answer and the verifier would cheerfully agree. Five independent derivations of the same rules means a silent error has to be made five times, in five notations, in the same way.

03 Independent verification

Each newer domain ships a verifier that **imports no solver and reuses no engine's transition code**. It re-derives every rule from the domain specification and reports every violation it finds, with a step index:

```
INVALID - 1 violation(s) across 1 category/categories: action
  [action] step 2: block 'B' is not clear; 'C' rests on it
```

Three mechanisms keep that independence from decaying:

- **A structural test** parses the verifier's abstract syntax tree and fails if any solver module appears among its imports.
- **The CLI refuses to emit** a result that fails verification, whichever engine produced it — exit code 3.
- **Renderers refuse to draw** an unverified plan. A picture of an impossible plan is worse than no picture at all.

Evidence is also explicitly ranked, and the test suites are ordered to match. Two engines agreeing with each other is the **weakest** signal available, because both could share a blind spot, so it is tested last.

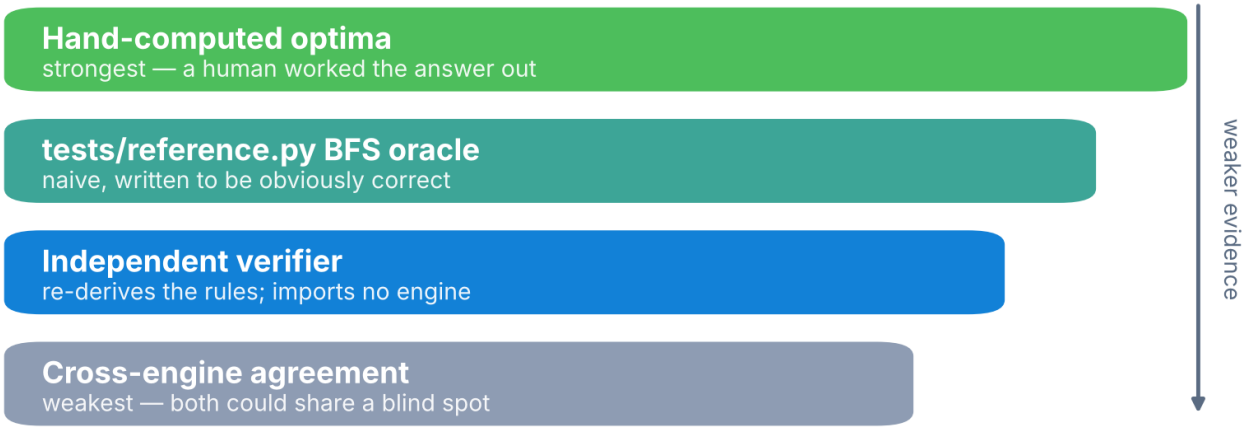


Figure 2. *The evidence hierarchy. Trident checks against hand-computed optima first and cross-engine agreement last — the reverse of the convenient order.*

04 The four problem domains

4.1 Cryptanalysis Toolkit — Haskell

FUNCTIONAL PARADIGM · GHC 9.4 · BASE ONLY · 25-CASE SUITE

Caesar and Vigenère ciphers with two working attacks: chi-squared frequency analysis to recover an unknown Caesar key, and Kasiski examination to estimate a Vigenère key length. Ciphers are pure transformations, decryption is encryption with a negated key, and the key stream is an infinite lazy list — the functional idiom doing real work rather than decorating an imperative algorithm.

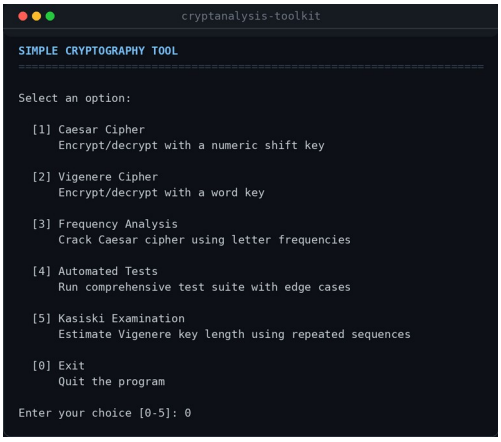
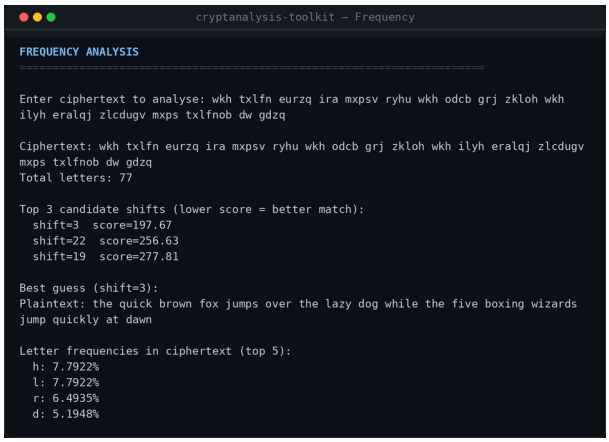




Figure 3. *The Haskell toolkit's menu-driven interface, and its 25-case built-in test suite running to completion.*



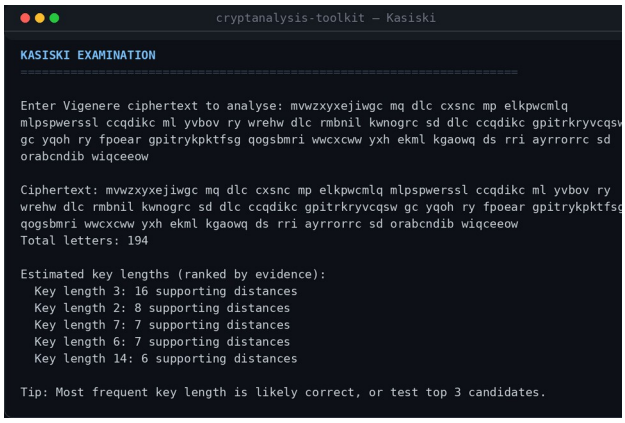


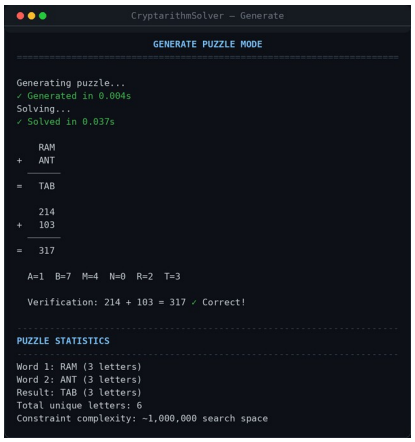
Figure 4. *Chi-squared frequency analysis recovering a Caesar key, and Kasiski examination estimating a Vigenère key length.*

4.2 Cryptarithm Solver — Python + Z3

CONSTRAINT PARADIGM · SMT WITH A BRUTE-FORCE CONTROL · 9-CASE SUITE

SEND + MORE = MONEY as a constraint problem, with a brute-force fallback that independently confirms the SMT encoding. Both engines agreed on all nine test puzzles. The measured speedup is the interesting part, because it is **not** a constant:

Puzzle	Distinct letters	Z3 vs brute force
SEND + MORE = MONEY	8	**350× faster**
A + B = C	3	**0.02× — i.e. 50× slower**



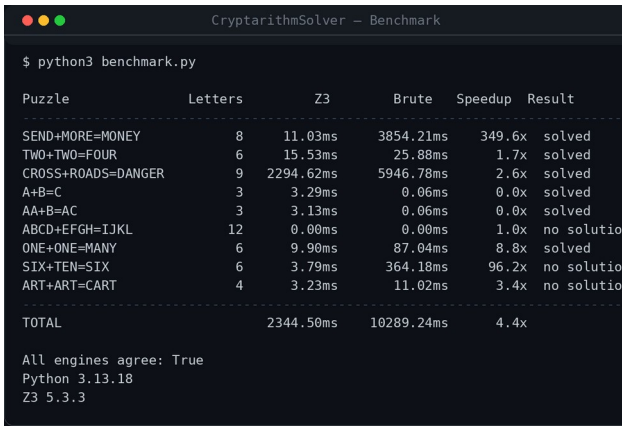


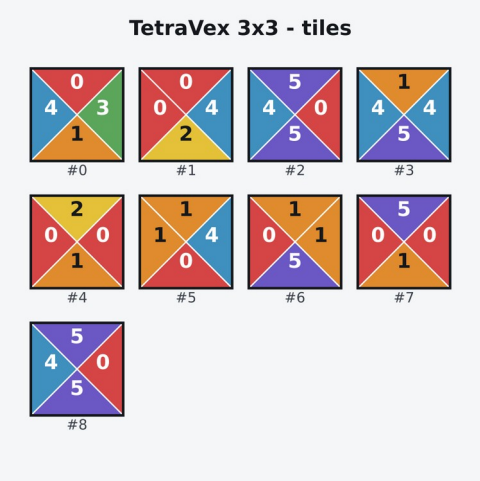
Figure 5. *The solver generating and solving a cryptarithm, and the benchmark harness comparing the Z3 encoding against the brute-force control.*

Below a certain search-space size, building the constraint system costs more than simply checking a few hundred permutations. This is the same lesson the newer domains deliver at larger scale, and it is the reason the project reports ranges and crossover points rather than single headline multipliers.

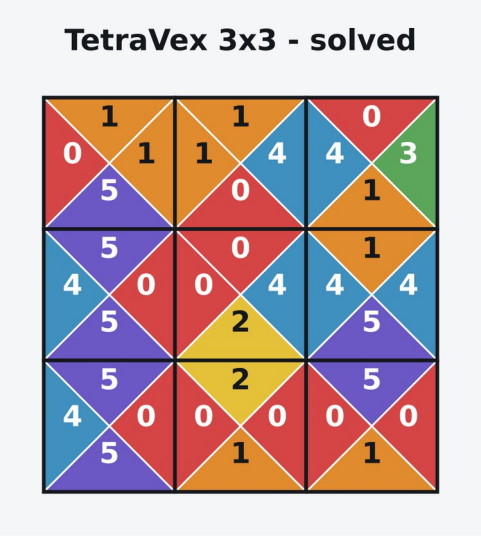
4.3 TetraVex — Z3 versus Prolog CLP(FD)

TWO ENGINES · ONE MODEL · IDENTICAL CONSTRAINT ORDER

Square tiles carrying a symbol on each edge must fill a grid so that every pair of touching sides matches. Both engines post the same five constraints in the same order, so the comparison is genuinely of solvers rather than of two different models. The implementation also supports rotation and a pinned border, and answers whether a solution is **unique** — Z3 by adding a blocking clause and re-solving, Prolog via findall/3 over limit(2, …).



Scrambled tile tray



Solved board

Figure 6. *SVG rendering is built in; the renderer refuses to draw any board that has not passed independent verification.*

4.4 Blocks World — symbolic search versus planning-as-SAT

TWO FUNDAMENTALLY DIFFERENT ATTACKS ON THE SAME PLANNING PROBLEM

- **Prolog** — iterative deepening bounded by f = g + h, where h counts misplaced blocks. *Admissibility*: a move changes the support of exactly one block, so it reduces h by at most 1; therefore h never exceeds the true distance, and the first solution found is optimal.
- **Z3** — assert “there is a plan of exactly T steps”, starting at T = 0 and incrementing. Every step is forced to be a real move, so the first satisfiable horizon **is** the optimal plan length.

Both return the optimal three-move plan for the **Sussman anomaly**, the classic case that defeats naive subgoal-ordering planners — a useful landmark precisely because the wrong answer is a well-known one.

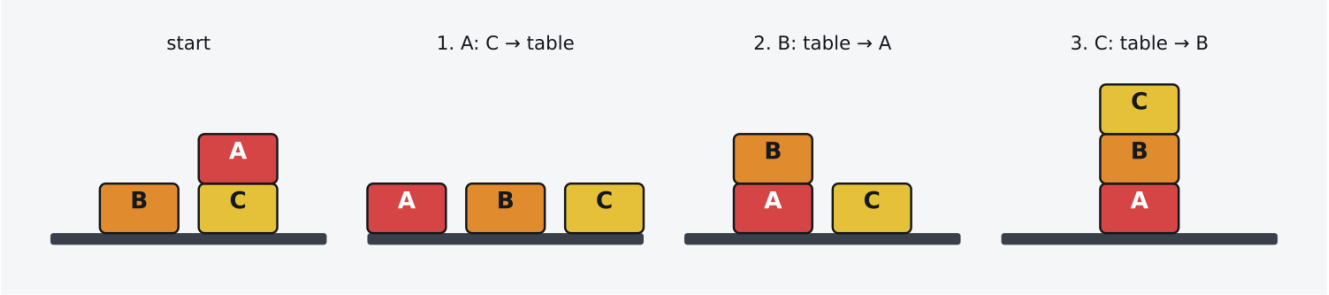


Figure 7. *The Sussman anomaly solved in three moves, rendered directly by the toolkit. ASCII, SVG, SMIL-animated SVG and GIF outputs are all supported.*

05 Results — and why the answer reverses

Every figure below comes from the repository's own benchmark harness. Both engines see identical seeded instances, every result counted as a success was independently verified first, and startup cost is calibrated separately so it cannot be mistaken for solve time.

TetraVex — Z3 wins, but not everywhere

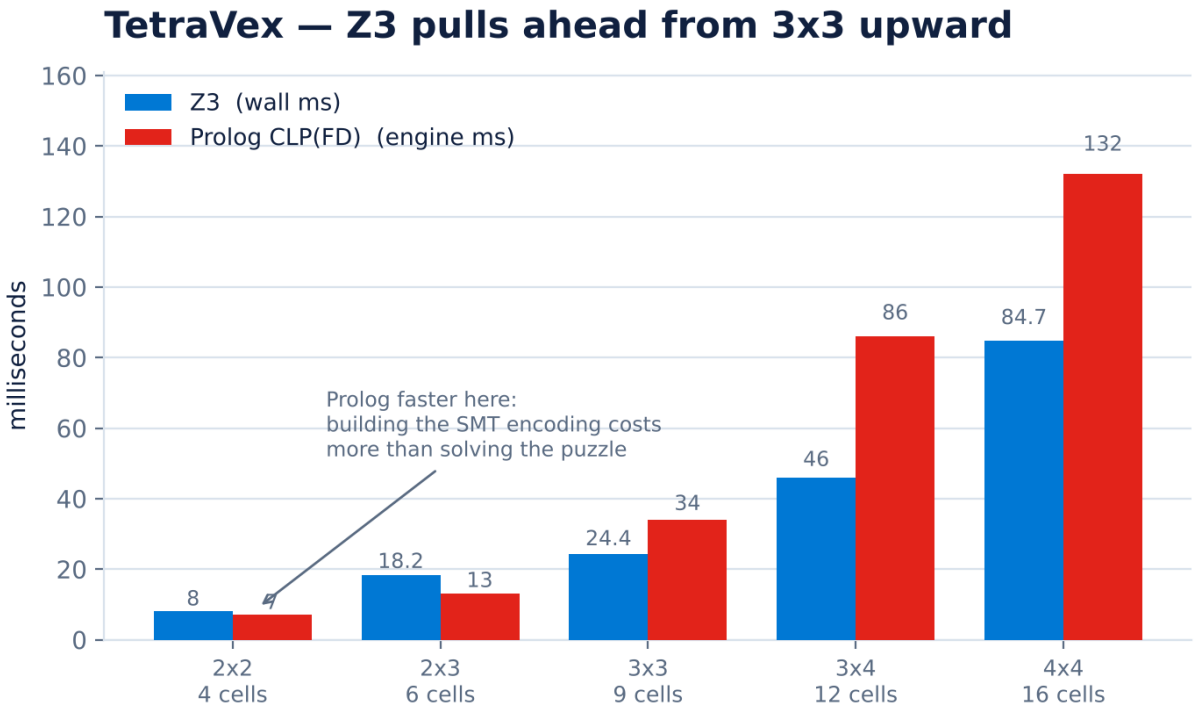


Figure 8. *6 symbols, free border, median of 3 repetitions over 3 seeds. Z3 engine time is not recorded by the harness, so Z3 is shown as wall time — a conservative comparison that favours Prolog.*

Prolog is genuinely **faster** on the two smallest boards — 0.88× at 2×2 and 0.71× at 2×3 — then Z3 pulls ahead from 3×3 upward and never gives the lead back. The crossover is the finding: an SMT solver's constant cost is real, and below it a lighter tool wins. The margin then widens with board size, reaching **7.6× at 5×5**.

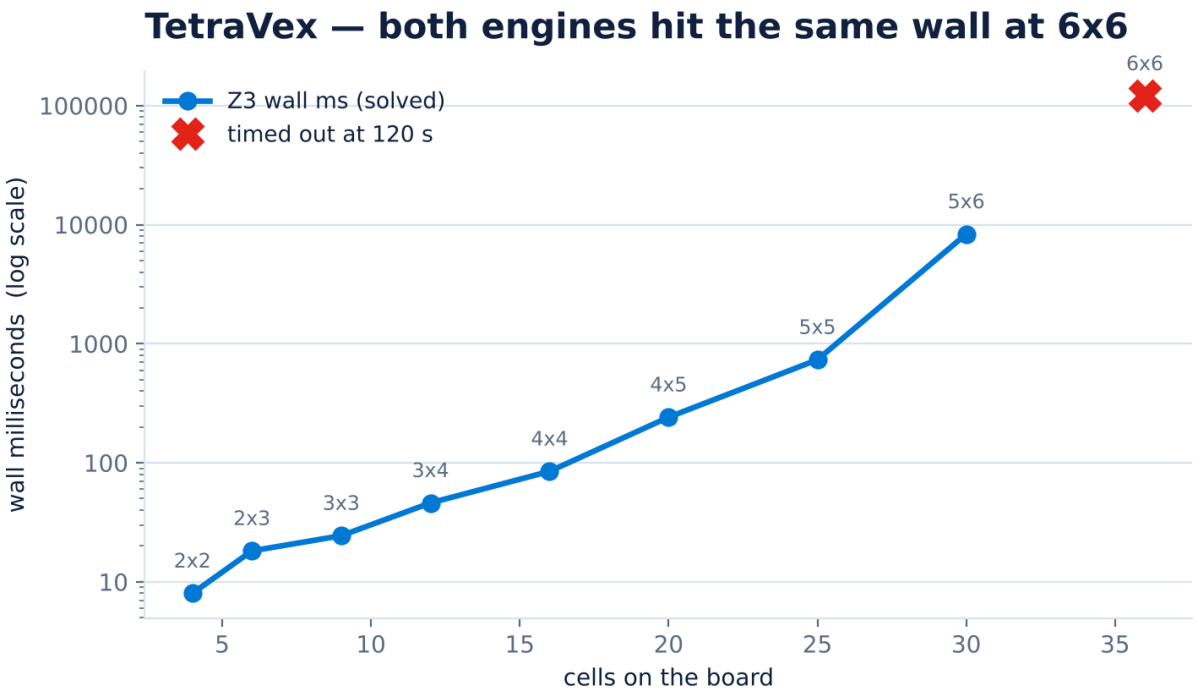


Figure 9. *Scaling to the wall. Both engines solve every board up to 5×6 (30 cells) and both time out at 6×6 against a 120-second limit — a shared ceiling, not an engine-specific one.*

Blocks World — Prolog wins, decisively

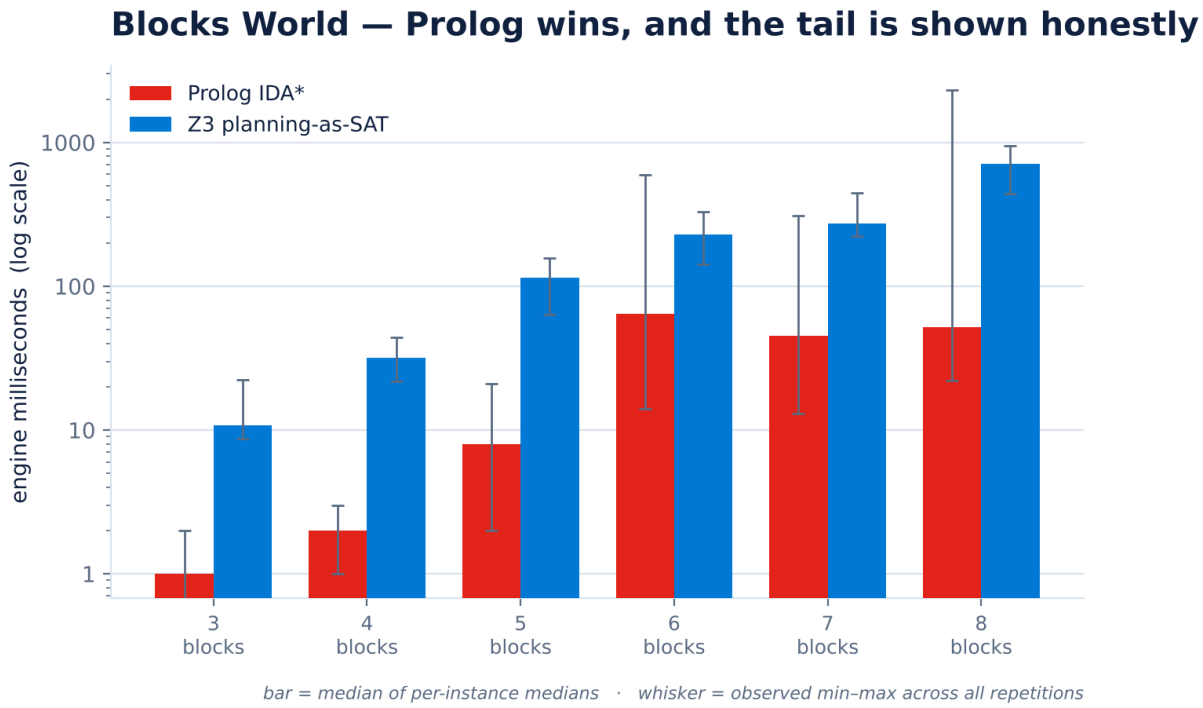


Figure 10. *3 seeds per size, 3 repetitions, two-stack instances. The engines agreed on plan length for every instance tested, and every plan passed independent verification.*

> ▲ **The honest caveat, kept in view**
>
> Prolog's 8-block range is **33–2223 ms**: one seed cost 67× another. Z3's spread is far tighter, which is its own kind of advantage — predictability. This is exactly why the project reports observed ranges rather than medians alone.

Why the reversal happens

To prove a 9-move plan optimal, Z3 must first prove horizons 0 through 8 unsatisfiable — each a fresh and larger constraint system. The per-horizon trace makes the cost structure visible:

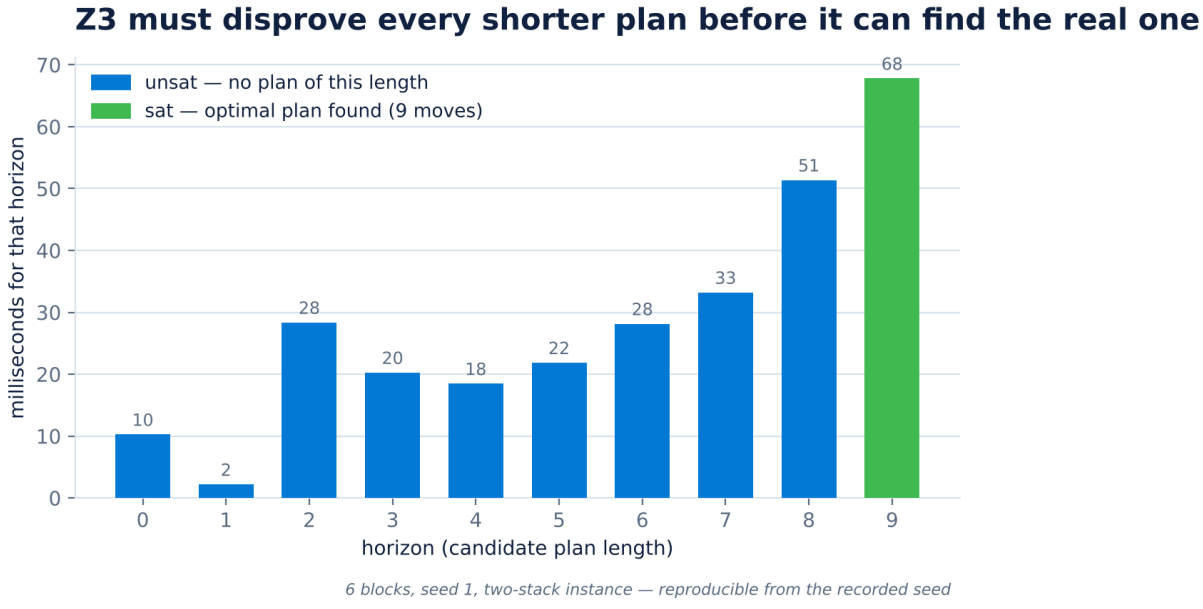


Figure 11. *Recorded per-horizon solve times for a 6-block instance. Nine of the ten solver calls are spent proving that shorter plans do not exist.*

Prolog's IDA* expands a few dozen states and stops. TetraVex is a single constraint system with no notion of sequence — exactly SMT's home ground. Blocks World is sequential, and asking “how long is the plan?” by trying every length in turn makes the solver pay repeatedly for something explicit search answers once.

The asymmetry is starkest when no plan exists

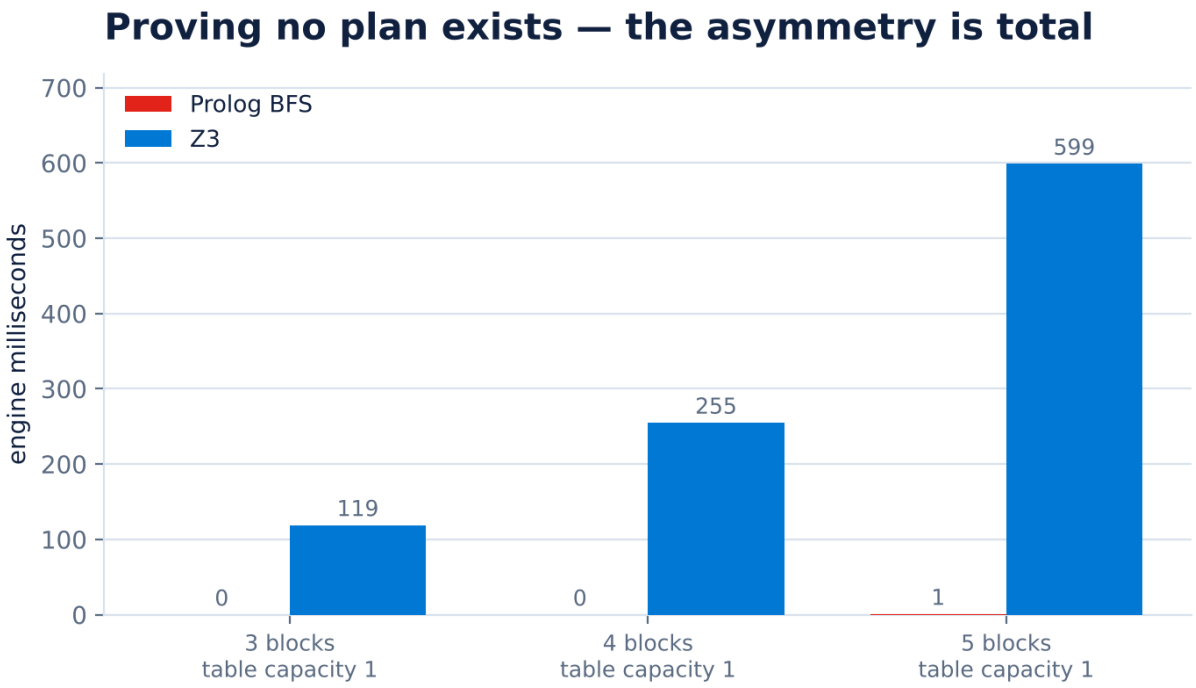


Figure 12. *Proving unsolvability on a table of capacity 1. Prolog's BFS settles it almost instantly; Z3's cost grows steeply because it must exhaust every horizon.*

limit is not unsat

An engine may report **unsat** only when it can justify it. “I searched to my ceiling and found nothing” is reported as **limit**, a third and distinct status. Collapsing the two is the easiest way for a planner to lie, and the distinction is enforced by tests.

06 Engineering: continuous integration and two instructive defects

The project runs seven CI jobs on every push, deliberately split so that a failure is attributable on sight rather than requiring a bisect.

CI job	What it proves	Status
lint	ruff across all project Python	**Passing**
python 3.10 – 3.13	Both suites against a **native** SWI-Prolog (4 jobs)	**Passing**
swipl-wasm fallback	Both suites with no native swipl on PATH	**Passing**
haskell	task1 compiles clean under -Wall; 25-case suite runs	**Passing**

The native and WASM split matters: the Prolog engines prefer a native binary and fall back to WebAssembly, and only running both proves that both paths work. Bringing that matrix to green surfaced two defects worth recording in full, because each was invisible to a different part of the safety net.

6.1 Defect — the native Prolog path never received its arguments

Field	Detail
Symptom	19 failed / 103 passed in TetraVex on **all four** Python versions
Exit signature	Prolog exited with code 1 — **empty stdout and empty stderr**
Scope	Every native Prolog call; the WASM path was unaffected
Files changed	4 (two Python drivers, two Prolog sources)

Root cause. The driver invoked swipl as `swipl … tetravex.pl <facts> <mode>`. SWI-Prolog treats that trailing `<facts>` path as a second **source file to consult**, not as an argument, so it never reached the argv flag. The entry point matched argv against `[File, ModeAtom|_]`, found only `[one]`, failed, and exited silently.

> ▲ **The subtlety that cost an extra debugging round**
>
> The separator `--` must come **after** the model file. Placed before it, the model stops being consulted as a script, `main/0` becomes undefined, and SWI-Prolog silently resolves `main` against `library(main)`'s `main/1` instead — a *different* failure (exit 2) wearing identical clothes.

THE FIX

```
# driver — the separator goes after the model, before the data
cmd = [binary, "-q", "-g", "main", "-t", "halt",
       str(_MODEL), "--", str(facts), mode]

% entry point — match the TAIL of argv, so it holds whether or not
% the binary's own name is included in the list
main :-
    current_prolog_flag(argv, Argv),
    append(_, [File, ModeAtom], Argv),
    run(File, ModeAtom, Json),
    write(Json), nl.
```

Outcome. 311 tests now pass against native SWI-Prolog 9.0.4 in CI — a path the project had previously documented as untested, because no SWI-Prolog package was available in the original development environment.

6.2 Defect — a mode bit that broke every documented command

Field	Detail
Symptom	`./trident: Permission denied` — exit 126, all four Python versions
When	**After** all 311 tests passed; the very next step failed
Root cause	The dispatcher was committed with git mode **100644**, not 100755
Origin	The bit is lost when a tree is rebuilt from an archive extracted on a filesystem with no Unix executable bit

■ Why the test suites could not catch this

The tests import the packages directly and **never go through the dispatcher**. So all 311 tests pass while every one of the 13 commands documented in the README is broken for anyone who clones the repository. A green suite proved nothing about the user-facing entry point.

The fix changes no file contents — only the mode recorded in the git index, which is why it works even where the filesystem cannot represent the bit:

```
git update-index --chmod=+x trident
```

The guard. Rather than work around the failure by invoking the dispatcher through the interpreter — which would have turned CI green while leaving users broken — the check was made explicit and fast. The lint job now inspects the committed mode directly and prints the remedy:

```
mode=$(git ls-files -s trident | cut -d' ' -f1)
if [ "$mode" != "100755" ]; then
  echo "::error file=trident::trident is committed with mode $mode, expected 100755"
  echo "    git update-index --chmod=+x trident" >&2
  exit 1
fi
```

◆ The transferable lesson

Both defects were invisible to a passing test suite, and in opposite ways: the first because a failure path produced **no diagnostic output at all**, the second because the tests exercised a different entry point than users do. Green tests describe what is tested — never what is shipped.

07 Quality assurance

122	189	34	0
TetraVex tests	Blocks World tests	Haskell + Z3 legacy	lint findings

Suite	Result	Runtime
ruff check .	**All checks passed**	—
task3-tetravex	**122 passed**, 0 skipped	~25 s
task4-blocksworld	**189 passed**, 0 skipped	~23 s
task1 Haskell suite	25 cases, 0 failed	—
task2 solver suite	9 cases, both engines agree	—

Solver tests **skip rather than fail** when a toolchain is absent, so a run reporting skips means the environment is incomplete, not that the code is sound. Both suites run with zero skips in CI, on both the native and the WebAssembly Prolog paths.

Benchmark methodology

- Both engines see **identical instances** generated from recorded seeds.
- Every result counted as a success is independently verified **before** it is counted.
- **Startup cost is calibrated separately** on a trivial instance, rather than subtracted from measurements where it would produce unstable or negative values.
- Two clocks are reported — wall (end-to-end) and engine (self-reported solve time). Engine is the figure compared.
- Medians with **observed ranges**, so a long tail stays visible rather than being averaged away.
- Search-effort columns stay in each engine's own units. States expanded and horizons attempted are **not** converted into a ratio, because no honest conversion exists.

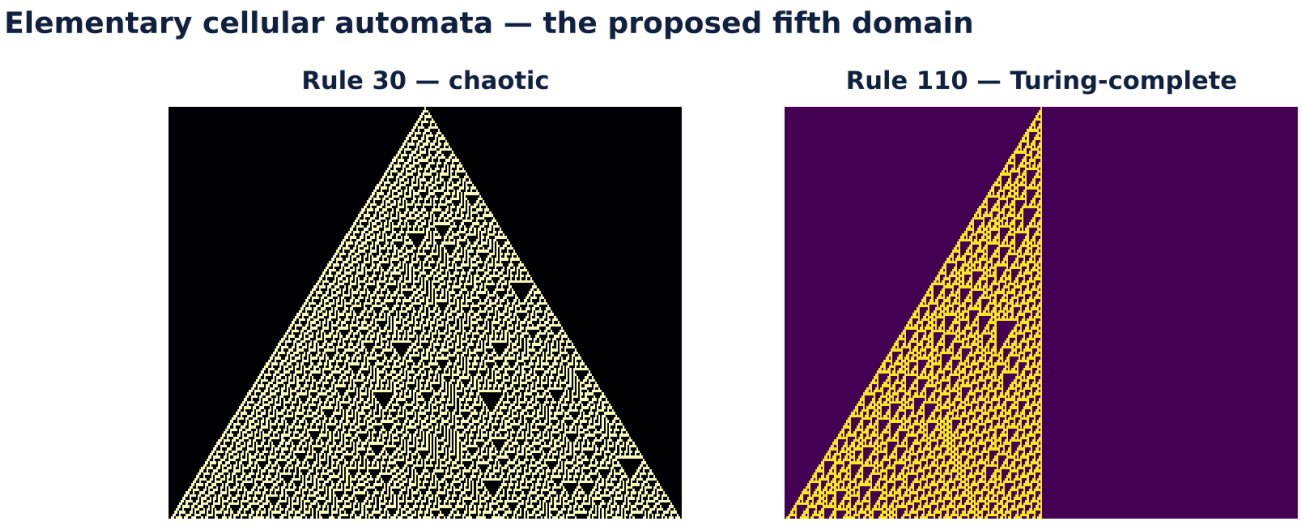
08 Known limitations

Stated plainly, because a comparative study that hides its weak points is not evidence of anything.

#	Limitation
1	**Published Prolog timings were measured through swipl-wasm.** The native path is now fully tested in CI, but the benchmark figures in this report predate that and have not been re-measured natively.
2	**Small samples** — 3 seeds × 3 repetitions. Medians and ranges are reported; no confidence intervals, because quoting one from three samples would be noise.
3	**Short plans only** (≤ 12 moves). The literature finds SAT planning competitive at much longer horizons; nothing measured here contradicts that.
4	**Blocks World goals must be complete configurations.** Partial goals are not supported.
5	**No sound unsolvability bound on a restricted table.** Where table capacity is below the block count, only BFS settles the question, and only while the state space fits in memory.
6	**TetraVex uniqueness ignores rotational duplicates** under the rotation variant.
7	**Preserved coursework is deliberately not modified**, because the published figures were measured against exactly that code. Its known defects are documented rather than fixed.

09 Roadmap — the fifth domain

The original assessment brief listed five problems. Trident implements four. **Cellular Automata simulation is the one not yet built**, and it is the most interesting remaining candidate — not because it is difficult to simulate, but because of what happens when the question is turned around.



Illustrative figure generated for this report. No cellular-automata engine exists in Trident today.

Figure 13. *Elementary cellular automata: Rule 30, whose output is chaotic enough to have been used as a random number generator, and Rule 110, which is proven Turing-complete. The structure these rules generate from a single live cell is the behaviour a solver would have to reason backwards through.*

Why it fits the comparison, rather than merely extending the table

Forward simulation of a cellular automaton is trivial in every paradigm and would prove nothing. The **inverse** problem is the valuable one:

> ◆ **The pre-image problem**
>
> Given a target configuration at generation *t*, does a configuration at *t−1* exist that produces it — and is it unique? Some states have many predecessors, and some have **none at all** (the "Garden of Eden" states). This is a genuine constraint satisfaction problem, not a simulation.

That framing maps directly onto the comparison this project already makes, and it would **test** the central thesis rather than merely illustrate it:

Question	Shape	Predicted winner	Why
Does a predecessor exist?	One static constraint system	**Z3**	Same shape as TetraVex — no notion of sequence
Shortest path to a target state	Sequential search, unknown length	**Prolog**	Same shape as Blocks World — horizon iteration is costly

A prediction that can be checked is worth far more than another benchmark row. If Z3 wins the pre-image problem and Prolog wins shortest-path, the thesis survives a genuine test on a domain it was not derived from. If not, the thesis needs revising — which is just as useful a result.

The scaffolding is already shaped for it

- **model.py** — a configuration is a bit vector; a rule is an integer 0–255. Simpler than either existing domain.
- **generator.py** — seeded random configurations, with rule and width as difficulty controls.

- **verify.py** — unusually strong here. To check a claimed predecessor, simply **simulate it forward one step** and compare. The verification method is utterly unlike the solving method, which is exactly the independence the project demands everywhere else.
- **render.py** — a spacetime diagram is a pixel grid; the existing SVG and GIF renderers transfer almost directly.
- **bench.py** — the harness is domain-agnostic and needs only a new suite definition.

Also on the roadmap

- **Partial goals in Blocks World** — the single change that most expands what the domain can express, and it lengthens optimal plans, which is precisely the regime where SAT planning is reported to become competitive.
- **Native-Prolog benchmark figures**, replacing the WebAssembly wall-clock numbers now that the native path is verified in CI.
- **A third engine per problem** — a hand-written backtracking solver as a control that is neither SMT nor CLP(FD).
- **CLP(FD) labelling strategies** (ff, ffc, min) compared on a fixed model, isolating search heuristics from modelling.

10 Technology stack

Paradigm	Technology	Where it is used
Functional	Haskell — GHC 9.4, base only	task1 — ciphers as pure transformations
SMT / declarative constraints	Z3 5.3.3 via z3-solver	task2, task3, task4
Constraint logic programming	SWI-Prolog 9.0.4, library(clpfd)	task3 — TetraVex model
Symbolic state-space search	SWI-Prolog — IDA*, IDDFS, BFS	task4 — Blocks World planner
Portable fallback	swipl-wasm 3.8.2 under Node	Both Prolog engines, when no native binary exists
Imperative baseline	Python brute force	task2 — control for the SMT encoding
Tooling	pytest · ruff · GitHub Actions · Pillow	Tests, lint, CI, GIF rendering

Selected references

- Wielemaker, J., Schrijvers, T., Triska, M. & Lager, T. (2012). SWI-Prolog. *Theory and Practice of Logic Programming*, 12(1–2), 67–96.
- de Moura, L. & Bjørner, N. (2008). Z3: An Efficient SMT Solver. *TACAS 2008*, LNCS 4963, 337–340.
- Kautz, H. & Selman, B. (1992). Planning as Satisfiability. *ECAI-92*.
- Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1), 97–109.
- Hart, P. E., Nilsson, N. J. & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- Régin, J.-C. (1994). A Filtering Algorithm for Constraints of Difference in CSPs. *AAAI-94*, 362–367.
- Takenaga, Y. & Walsh, T. (2006). Tetravex is NP-complete. *Information Processing Letters*, 99, 171–174.

- Gupta, N. & Nau, D. S. (1992). On the complexity of blocks-world planning. *Artificial Intelligence*, 56, 223–254.
- Wolfram, S. (2002). *A New Kind of Science*. Wolfram Media. — cellular automata rule space.
- Cook, M. (2004). Universality in Elementary Cellular Automata. *Complex Systems*, 15(1), 1–40. — Rule 110.

In summary

Trident set out to answer one question — when two solver paradigms can both attack a problem, which wins? — and the answer it measured is that the question is incomplete without naming the problem's shape. A static constraint system favours SMT; a search for a sequence of unknown length favours explicit search. The same two engines reverse places between TetraVex and Blocks World, and the per-horizon trace in Figure 11 shows exactly where the cost goes.

◆ What the project demonstrates

Comparative benchmarking designed so that its own results can be falsified — independent verification, seeded reproducibility, three distinct solver statuses, and measurements reported with their tails intact. Plus the discipline to keep two defects on the record rather than quietly fixing them: **a green test suite describes what is tested, never what is shipped.**
